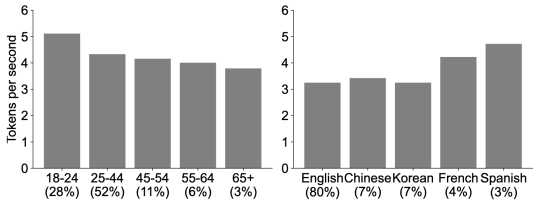
Andes and TokenFlow

This report compares the similarities between Andes and TokenFlow. We find numerous parallels in figures, problem formulation, motivating examples, evaluation structure, and language used in text.

Token Consumption Speed (TokenFlow Figure 1, Andes Figure 2)

Andes Figure 2



(a) ***Reading speed and age group*** **(b)** ***Speaking speed and language***

Figure 2: ***User token consumption speed based on demographic group, counting one word as 1.3 tokens on average [25].***

of decision-making and incorporates the overhead of request preemption and restart in its scheduling decision.

Andes's server implements push-based streaming, which immediately transmits tokens to the client as they are gen-

TokenFlow Figure 1

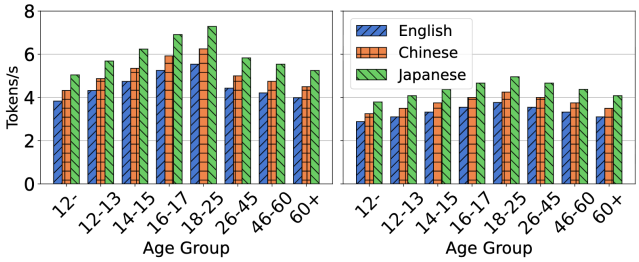
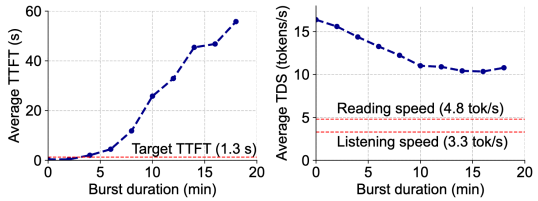


Figure 1. We summarize the token consumption speeds for reading (left) and for listening (right), measured across different age groups and language users. The data is derived from calculations based on reading speed data from NIH [31] and information on token counting from OpenAI's blog [35].

TokenFlow's Figure 1 and Andes's Figure 2 present nearly identical data in nearly identical format: both show user token consumption speeds (in tokens/sec) across age groups for reading and across languages for listening/speaking. This shared figure reflects the common motivation underlying both papers: that human consumption rates are far slower than LLM generation speeds, creating optimization opportunities.

Burst Load Analysis (TokenFlow Figure 2, Andes Figure 4)

Andes Figure 4



(a) ***Average TTFT*** **(b)** ***Average TDS***

Figure 4: ***vLLM's average TTFT and TDS while varying the duration of the load surge in our 20-minute trace. TTFT target is set to 1.3 s, as recommended by Google for web page loading [5]. User reading/listening speeds are 4.8 and 3.3 tokens/s, derived from Figure 2. TTFT is inflated significantly, whereas TDS remains excessively fast compared to any token consumption speed.***

TokenFlow Figure 2

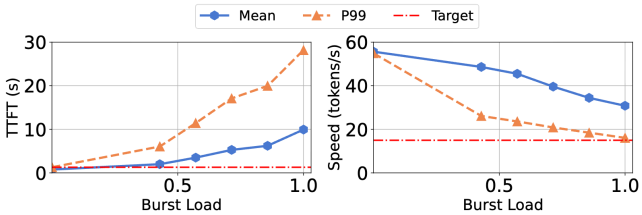


Figure 2. Micro-benchmark on SGLang's burst request handling conducted on the single NVIDIA H200 GPU. Left: Time-to-First-Token (TTFT) surges beyond acceptable thresholds (1.3s, red line) under increasing request intensity. Right: Generation speed declines but remains excessively high (2× average reading speed for reference, red line).

TokenFlow's Figure 2 and Andes's Figure 4 both demonstrate the same fundamental problem with existing LLM serving systems under load again in nearly identical format: TTFT spikes well beyond acceptable thresholds (both use 1.3s as the target) while token generation speed remains excessively high relative to user consumption rates. Both figures motivate their respective papers by showing this resource misallocation: over-generating tokens for active requests while starving queued requests.

User Experience Factors (TokenFlow Figure 5, Andes Figure 5)

Andes Figure 5

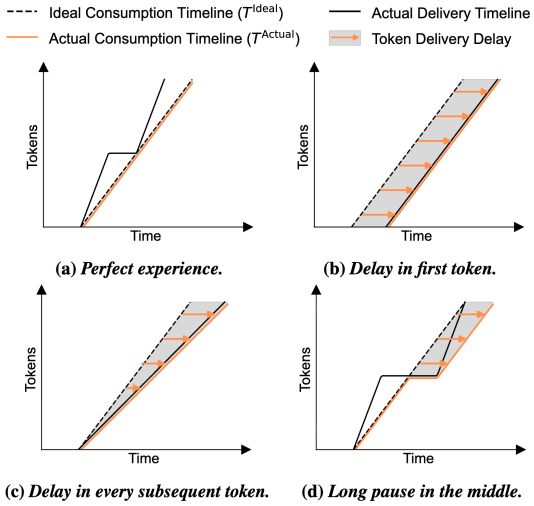


Figure 5: *User experience examples. Users expect tokens to be*

TokenFlow Figure 5

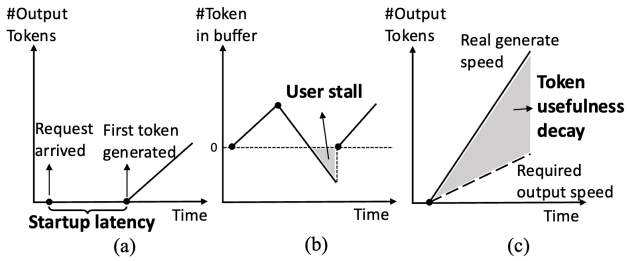


Figure 5. Three QoS factors: (a) Startup latency, (b) User stall events, (c) Token usefulness.

TokenFlow's Figure 5 and Andes's Figure 5 use the same visual framing to succinctly illustrate a request's token timeline: cumulative output token count on the y-axis versus time on the x-axis, with an Ideal Consumption Timeline (dashed line) compared against an Actual Delivery/Consumption Timeline. Both use shaded regions to highlight the discrepancy between ideal and actual, representing degraded user experience. Both figures convey that startup delay and mid-stream stalls degrade experience, while tokens delivered beyond consumption capacity provide diminishing value. To the best of our knowledge, Andes was the first to produce this style of visualization.

System Architecture (TokenFlow Figure 4, Andes Figure 6)

Andes Figure 6

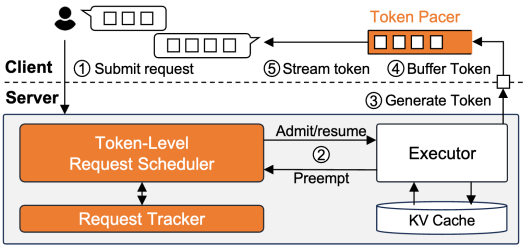


Figure 6: *High-level architecture of Andes. Components designed by Andes are colored in orange.*

TokenFlow Figure 4

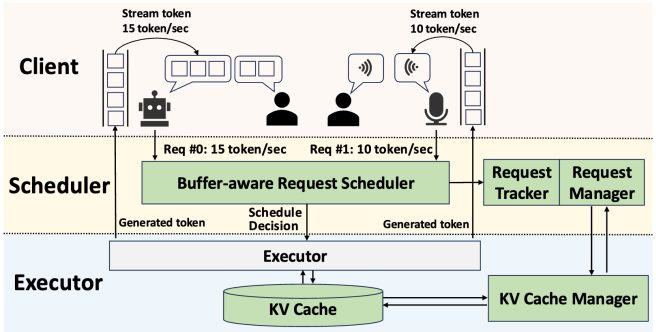


Figure 4. High-level workflow of TokenFlow. Modules newly added by TokenFlow are colored green.

TokenFlow's Figure 4 and Andes's Figure 6 are both high-level system architecture diagrams with similar structures: a client-server design featuring a specialized request scheduler (Buffer-aware vs. Token-Level), a Request Tracker, an LLM Executor, KV Cache management, and client-side token buffering (token buffer

vs. Token Pacer). The two systems share the same core components and data flows, differing mainly in nomenclature. Both figures use similar language in their captions to highlight novel components:

- Andes: "Components designed by Andes are colored in orange"
- TokenFlow: "Modules newly added by TokenFlow are colored green."

Evaluation Structure (TokenFlow Section 7, Andes Section 6)

Both evaluations follow the same structure: (1) real-world trace replay using BurstGPT, (2) synthetic/controlled workloads, (3) sensitivity analysis and ablation studies.

For real-world traces, both show temporal variation of queued and running requests over time comparing their system to baseline:

Andes Figure 3 + 13

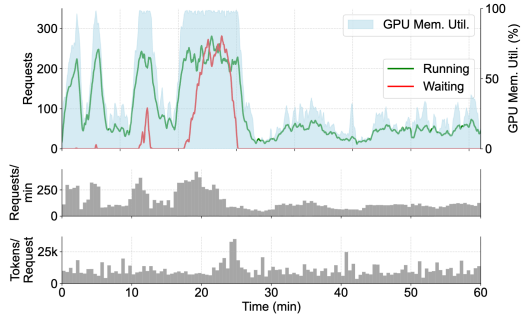


Figure 3: ***vLLM serving requests from BurstGPT. During load surges, vLLM builds up a very long queue due to head-of-line blocking. Still, under normal load, GPU memory is underutilized.***

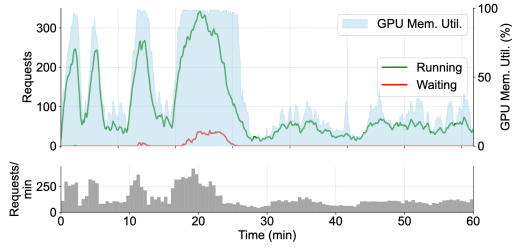


Figure 13: ***Andes serving requests from BurstGPT. vLLM serving the same trace is shown in Figure 3. Queue length is significantly reduced under load surges, and GPU memory utilization is higher.***

TokenFlow Figure 11 + 14 + 15

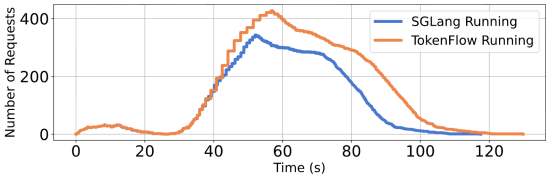


Figure 15. Temporal variation of **running requests** during a representative trace segment.

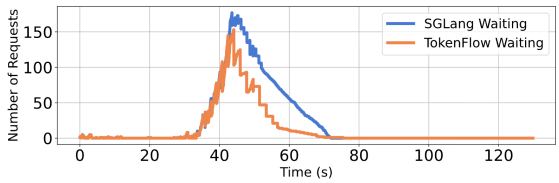


Figure 14. Temporal variation of **queued requests** during a representative trace segment.

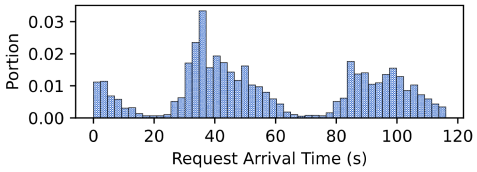
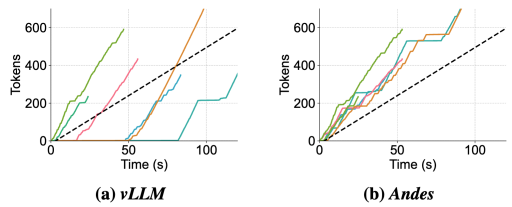
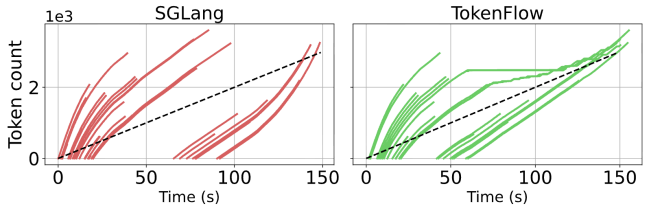


Figure 11. The distribution of the real-world trace.

Andes Figures 3 (baseline) and 13 (Andes) combines request arrival, running requests, and waiting requests in one figure; TokenFlow splits these across Figures 11, 14, and 15.

Both also show token generation timelines for individual requests, plotting cumulative tokens vs. time with an ideal/required consumption line, comparing their system (timelines stay above the line) to baseline (timelines fall below):

Andes Figure 12	TokenFlow Figure 18
 **Figure 12:** ***The Actual Delivery Timeline of select requests, with request submission time shifted to zero. Requests served by Andes, as opposed to vLLM, are well above the Ideal Consumption Timeline.***	 **Figure 18.** Token generation timelines comparing SGLang (left) and TokenFlow (right). TokenFlow maintains generation speeds consistently above requirements (black line) while achieving lower average TTFT.

Problem Formulation (TokenFlow Section 3.3, Andes Section 4.1)

Both formulations structure the scheduling problem nearly identically: solved per scheduling timestep, binary decision variables x_i (same notation) for each request, a constraint on GPU memory M (same notation), a constraint on batch size B (same notation), and an objective to maximize the sum of per-request utility. Both are knapsack variants operating in an online setting with dynamic request arrivals. The per-request "value" differs in form but both capture the tradeoff between serving a request now versus leaving it waiting. TokenFlow incorporates context-switch overhead upfront into the formulation, while Andes incorporates preemption overhead into the formulation later in Section 4.3.

Motivating Example (TokenFlow Section 4.1, Andes Section 4.2)

Both papers present a toy example illustrating the same scenario:

1. initial requests run and accumulate buffer,
2. new requests arrive causing overload,
3. the request with the largest buffer is preempted to make room,
4. the preempted request resumes when its buffer nears depletion.

Andes uses token counts and TokenFlow uses token rates in their illustrations, but the scenario structure and scheduling pattern are the same.

Implementation (TokenFlow Section 6, Andes Section 5)

Both sections follow the same structure: an opening sentence listing main components, then subsections for each. The scheduler subsections use nearly identical phrasing:

- Andes: "Andes's scheduling algorithm can work with any LLM serving system that supports continuous batching and at least one preemption mechanism"
- TokenFlow: "Our scheduling algorithm works with any inference framework that supports basic preemption"

TokenFlow's Claims About Andes (TokenFlow Section 8)

TokenFlow Section 8 claims: "While Andes primarily emphasizes user-perceived Quality of Experience (QoE), it overlooks the broader perspective of overall inference server performance and resource utilization" and "Andes lacks such coordinated mechanisms."

These claims go too far. Andes's objective (maximizing request rate while keeping the same QoE) is optimizing resource utilization, and Andes's scheduling decisions explicitly considers runtime memory and compute constraints. Andes also has coordination between scheduling and memory management through its overhead-aware refiner (Section 4.3), which feeds preemption overhead estimates back into scheduling

decisions. The valid distinction is that Andes uses reactive memory management while TokenFlow uses proactive write-through KV cache management.